FSM widths and counter connections

The width warnings come from assigning wide constants to one-bit outputs. The counter question is about following a signal through a parent module and its child instance. Both become easier when you write down each signal's width and driver.

Where this fits in your sheet

Entries 102, 107, 109. Day 6, Day 7.

Entry numbers follow the tracker. Day labels in older filenames refer to when the notes were written; use the entry numbers to find the matching problem.

Pages	What to read
2-3	Entry 102: Quartus truncation warnings and Y0 decoding
4	Entry 107: simplifying Y2 next-state logic
5	Entry 109: Q, c_enable, c_load and c_d
6-7	Short reviews of entries 99-110 and source links

Reviewed against the saved tracker and available code on 7 September 2026. Earlier submission dates inside these notes are historical records.

Completion map

Entry	Problem	Core concept	Notes
99	Even longer vectors	Adjacent-bit vector operations and wraparound	No question
100	Decade counter again	Synchronous 1-through-10 counter	No question
101	Modules	Instantiate a provided module	No question
102	Q3c: FSM logic	Derive Y[0] and Moore output z	Width-warning question
103	2-to-1 multiplexer	One-bit conditional select	No question
104	Slow decade counter	Clock-enable behavior and synchronous reset	No question
105	Connecting ports by position	Positional port mapping	No question
106	Sequential circuit 7	Positive-edge DFF implementing q <= ~a	No question
107	Q6b: FSM next-state logic	Derive next value Y2 from the state graph	Width-warning question
108	2-to-1 bus multiplexer	100-bit conditional select	No question
109	Counter 1-12	Hierarchical counter control and observable ports	Interface question
110	Connecting ports by name	Named port mapping	No question

Question-bearing entries are highlighted in the tracker with a direct link to this PDF. The other nine entries are recorded as **No questions asked**.

Why Quartus Warning 10230 appears

Kapil saw the same warning pattern in entries 102 and 107: **truncated value with size 32 to match size of target (1)**. Quartus is describing a width conversion, not a failed simulation.

In Verilog, a plain decimal literal such as `0` or `1` is unsized and is treated with integer width. In these expressions Quartus carries a 32-bit value, but `Y0` and `Y2` are one-bit outputs. Assignment keeps the least-significant bit and discards the other 31 bits, so Quartus warns.

Expression	Expression width	Target	Result
Y2 = 0;	32-bit unsized integer	1-bit Y2	0 is kept; 31 zero bits discarded
Y2 = (w) ? 1 : 0;	32-bit ternary result	1-bit Y2	LSB is correct, but warning remains
Y0 = 3'b000;	3-bit sized literal	1-bit Y0	1 LSB kept; two bits discarded
Y2 = 1'b0;	1-bit sized literal	1-bit Y2	Exact width; no truncation
Y2 = w;	1-bit signal	1-bit Y2	Exact width; simplest form

The accepted results are logically correct because truncating 32-bit zero or one leaves the expected low bit. Still, the clean fix is to match widths deliberately: use `1'b0`/`1'b1`, or assign the already-one-bit Boolean signal directly. This makes the designer's intent explicit and prevents a harmless warning from hiding a later harmful truncation.

Intel's warning reference says the target is too small for the assigned value and recommends increasing the target width or decreasing the assigned value width when the truncation is not intended.

Entry 102 - Q3c: derive Y0 and z

Kapil's saved question: **Why do the Quartus truncated-value messages appear?**

The problem supplies the full next-state table. Only bit 0 of the next-state vector is required. Read the rightmost bit of each next-state code; do not build a state register here.

Present y	Next state x=0	Y0 when x=0	Next state x=1	Y0 when x=1	z
000	000	0	001	1	0
001	001	1	100	0	0
010	010	0	001	1	0
011	001	1	010	0	1
100	011	1	100	0	1

Rows 000 and 010 produce `Y0 = x`. Rows 001, 011, and 100 produce `Y0 = ~x`. Output z depends only on the present state, so it is a Moore output: it is high in states 011 and 100.

In Kapil's accepted code, each ternary branch used unsized `0` and `1`, and the default used `3'b000`. Those are exactly the 32-to-1 and 3-to-1 width conversions reported by Quartus.

A warning-free version is shorter and follows the table directly:

```
module top_module (
    input        clk,
    input  [2:0] y,
    input        x,
    output reg   Y0,
    output       z
);
    always @(*) begin
        case (y)
            3'b000, 3'b010: Y0 =  x;
            3'b001, 3'b011,
            3'b100:         Y0 = ~x;
            default:        Y0 = 1'b0;
        endcase
    end

    assign z = (y == 3'b011) || (y == 3'b100);
endmodule
```

Why is `clk` unused? The exercise asks only for combinational logic functions `Y0` and `z`. A complete FSM would register `Y` into `y` on a clock edge, but that state-register block is intentionally outside this problem.

Latch check: every possible `case` path assigns `Y0`, including `default`. Therefore no storage is inferred.

Entry 107 - Q6b: derive the next value Y2

> Kapil's saved question: **Why does Quartus report six 32-to-1 truncation warnings?**

There is one warning for each state branch because every assignment uses a ternary whose results are unsized decimal 0 and 1. The state decoding is accepted; the issue is only the expression width.

State	y[3:1]	Saved branch	Simplified Y2
A	000	w ? 0 : 0	0
B	001	w ? 1 : 1	1
C	010	w ? 1 : 0	w
D	011	w ? 0 : 0	0
E	100	w ? 1 : 0	w
F	101	w ? 1 : 1	1

The first simplification is semantic: if both ternary branches are equal, the selector does not matter. If the branches are 1 and 0, the ternary equals the one-bit selector itself. That removes all six unnecessary ternaries.

```
module top_module (
    input  [3:1] y,
    input        w,
    output reg   Y2
);
    always @(*) begin
        case (y)
            3'b000, 3'b011: Y2 = 1'b0;
            3'b001, 3'b101: Y2 = 1'b1;
            3'b010, 3'b100: Y2 = w;
            default:        Y2 = 1'b0;
        endcase
    end
endmodule
```

Signal naming matters: lowercase `y[2]` is the present-state flip-flop output. Uppercase `Y2` is the combinational value that would be loaded into that flip-flop on the next active clock edge. The task asks only for `Y2`, not the register itself.

Safe-warning rule: do not suppress a width warning until you can identify exactly which bits are discarded. Here, the discarded bits are redundant sign-extension zeros from values 0 and 1. In a counter, bus, address, or arithmetic result, the same warning can indicate real data loss.

Entry 109 - understand the 1-12 counter interface

> Kapil's saved question: **Q is the output of the counter. This input/output declaration is new to me - explain it.**

The top module is not asked to implement the four flip-flops. HDLBits provides a reusable `count4` submodule. The top module must generate its three control inputs and expose both its count and those controls so the testbench can verify the hierarchy.

Top-level port	Direction	Who drives it	Purpose
clk	input	HDLBits testbench	Clock forwarded to count4
reset	input	HDLBits testbench	Requests synchronous load of 1
enable	input	HDLBits testbench	Allows normal counting
Q[3:0]	output	count4 instance	Visible current counter value
c_enable	output	top_module assign	Also drives count4.enable
c_load	output	top_module assign	Also drives count4.load
c_d[3:0]	output	top_module assign	Also drives count4.d; fixed at 1

An `output` without `reg` is a net in Verilog. That is exactly what `Q` needs here: the output port of the instantiated `count4` drives it. The top module does not assign `Q` in an `always` block.

The three `c_*` signals have two simultaneous roles. They leave `top_module` as observable outputs, and the same nets enter the provided counter as control inputs. One net can connect both destinations because it still has only one driver: the continuous assignment in `top_module`.

```
assign c_d      = 4'd1;
assign c_load   = reset || (enable && (Q == 4'd12));
assign c_enable = enable;

count4 dut (
    .clk    (clk),
    .enable (c_enable),
    .load   (c_load),
    .d      (c_d),
    .Q      (Q)
);
```

Before edge	reset	enable	c_load	Action at edge	Q after edge
Q = 7	0	0	0	Hold	7
Q = 7	0	1	0	Increment	8
Q = 11	0	1	0	Increment	12
Q = 12	0	1	1	Load c_d	1
Any Q	1	any	1	Load c_d (load priority)	1

The provided counter checks load before enable. Reset therefore loads 1 even with enable low. At Q=12, wrapping needs enable=1; otherwise Q holds 12. The next-state equations use the value before the edge.

Using `|` instead of `||` is also valid for these one-bit control signals. `||` expresses Boolean intent; `|` emphasizes bitwise hardware. The important point is that both operands are one bit and the load-priority behavior comes from `count4`.

Quick review of the other nine completions

Entry	Key idea	Revision point
99	Adjacent vector bits	Slices can replace loops: out_both = in[98:0] & in[99:1]; out_any = in[99:1] | in[98:0]. For wraparound XOR, align {in[0], in[99:1]} against in.
100	Counter 1 through 10	Use nonblocking <= in a clocked always block. Reset to 1; when the present count is 10, load 1; otherwise increment.
101	Single submodule	Instantiation creates hardware hierarchy. The instance name is required and positional connections follow the child module declaration order.
103	One-bit mux	The conditional operator maps directly to a 2-to-1 mux: sel ? b : a.
104	Slow-enable counter	Only change q when reset or slowena is asserted. An explicit q <= q hold branch is unnecessary in a clocked block.
105	Ports by position	Short but fragile: any change in the child port order requires every positional instance to be checked.
106	Sequential circuit 7	q samples ~a on each rising edge. Between rising edges q holds its previous registered value.
108	100-bit mux	The same scalar select controls 100 parallel one-bit muxes; Verilog's vector conditional keeps the expression concise.
110	Ports by name	More verbose but order-independent. Match each .child_port(parent_signal) pair explicitly.

Preferred vector form for entry 99:

```
assign out_both      = in[98:0] & in[99:1];
assign out_any       = in[99:1] | in[98:0];
assign out_different = in ^ {in[0], in[99:1]};
```

Why the concatenation works: bit 99 of the shifted neighbor vector is `in[0]`, giving the required wraparound pair. Bit 98 is `in[99]`, bit 97 is `in[98]`, and so on through bit 0, whose left neighbor is `in[1]`.

Preferred sequential style for entries 100 and 104:

```
always @(posedge clk) begin
    if (reset)
        q <= RESET_VALUE;
    else if (enable_or_slowena)
        q <= (q == TERMINAL_VALUE) ? RESET_VALUE : q + 1'b1;
end
```

Nonblocking assignments model flip-flops clearly: every right-hand side is evaluated from the old pre-edge state, and updates become visible together after the edge. Omitting the final else branch intentionally means the register holds its value.

Revision checklist

- Size constants to the destination when width matters: 1'b0, 1'b1, 3'b000, and 4'd12.

- Simplify a ternary before coding: c ? 0 : 0 becomes 0; c ? 1 : 1 becomes 1; c ? 1 : 0 becomes c.

- For next-state logic, keep present state y separate from next-state value Y.

- Assign every combinational output on every path to prevent accidental latch inference.

- Use nonblocking assignments in edge-triggered always blocks.

- Treat a submodule output connected to a top-level output as a single driven net, not as a second procedural register.

- With named ports, read .child_port(parent_signal) from the inside out: the parent signal connects to that child port.

- Before ignoring a truncation warning, write down the source width, target width, and discarded bits.

Sources

HDLBits Gatesv100: https://hdlbits.01xz.net/wiki/Gatesv100

HDLBits Count1to10: https://hdlbits.01xz.net/wiki/Count1to10

HDLBits Module: https://hdlbits.01xz.net/wiki/Module

HDLBits Exams/2014 q3c: https://hdlbits.01xz.net/wiki/Exams/2014_q3c

HDLBits Mux2to1: https://hdlbits.01xz.net/wiki/Mux2to1

HDLBits Countslow: https://hdlbits.01xz.net/wiki/Countslow

HDLBits Module pos: https://hdlbits.01xz.net/wiki/Module_pos

HDLBits Sim/circuit7: https://hdlbits.01xz.net/wiki/Sim/circuit7

HDLBits Exams/m2014 q6b: https://hdlbits.01xz.net/wiki/Exams/m2014_q6b

HDLBits Mux2to1v: https://hdlbits.01xz.net/wiki/Mux2to1v

HDLBits Exams/ece241 2014 q7a: https://hdlbits.01xz.net/wiki/Exams/ece241_2014_q7a

HDLBits Module name: https://hdlbits.01xz.net/wiki/Module_name

Intel Quartus warning 10230 reference: https://www.intel.com/content/www/us/en/programmable/quartushelp/16.0/msgs/msgs/wvrfx_l2_veri_expression_truncated_to_fit.htm

IEEE 1364-2001 errata, expression bit-length table: https://standards.ieee.org/wp-content/uploads/import/documents/erratas/1364-2001_errata.pdf